

Express.js – Short Notes for Interviews

Quick revision sheet (Freshers: Full Stack / Web / Software Development)

1 Basics

- Express.js is a **minimal, flexible web framework** built **on top of** Node's `http` module. It simplifies routing, middleware, and request/response handling.
- Not a replacement for Node.js – it still uses `http` internally, just gives a nicer API.
- Setup:**

```
const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Hello World'));
app.listen(3000, () => console.log('Server running'));
```

- Routing pattern: `app.METHOD(PATH, HANDLER)` – methods: `get`, `post`, `put`, `patch`, `delete`.
- `res.send()` auto-detects content type (string/object/buffer); `res.json()` always sends JSON – prefer `json()` for APIs.
- Express is **unopinionated/minimal**: no forced project structure or tools.

2 Middleware

- A function (`req, res, next`) that runs **between** the request and the final response. Can run code, modify `req/res`, end the cycle, or call `next()`.
- Forgetting to call `next()` (and not sending a response) makes the request **hang forever**.
- Order matters**: middleware executes top to bottom, exactly as defined. Auth middleware must be placed *before* protected routes.

Type	Description
Application-level	<code>app.use()</code> / <code>app.METHOD()</code> – runs for matched requests
Router-level	Same, but bound to <code>express.Router()</code> instance
Built-in	<code>express.json()</code> , <code>express.urlencoded()</code> , <code>express.static()</code>
Third-party	<code>cors</code> , <code>morgan</code> , <code>helmet</code>
Error-handling	<code>(err, req, res, next)</code> – exactly 4 params

- `express.json()`: parses JSON body → `req.body`. Without it, `req.body` is `undefined`.
- `express.urlencoded({extended: true})`: parses HTML form data; `extended:true` allows nested objects (uses `qs`), `false` uses built-in `querystring`.
- `express.static(folder)`: serves static files directly (images, CSS, JS) without individual routes.
- Route-level middleware chaining: `app.get('/x', mw1, mw2, handler)`. If `mw1` sends a response, the rest are skipped.
- `next(error)`: skips all normal middleware/routes and jumps straight to **error-handling middleware**.

3 Routing (Detailed)

Aspect	Route Params	Query Params
Syntax	<code>/users/:id</code>	<code>/users?id=5</code>
Access	<code>req.params</code>	<code>req.query</code>
Purpose	Identify a specific resource	Filter/sort/search (optional)

- All values in `req.params` and `req.query` are always **strings**.
- `app.route(path).get().put().delete()` chains multiple methods for one path.
- `express.Router()`: mini mountable app for modular routes.

```
// routes/userRoutes.js
const router = express.Router();
router.get('/', getUsers); // -> /users
router.get('/:id', getUser); // -> /users/:id
module.exports = router;

// app.js
app.use('/users', require('./routes/userRoutes'));
```

- Good structure: **Routes** → **Controllers** → **Models** → **Middleware** (separation of concerns, MVC-like).
- First matching route (in definition order) handles the request.

4 Request & Response Objects

req property	Meaning
req.params	Route parameters
req.query	Query string parameters
req.body	Body data (needs <code>express.json()/urlencoded()</code>)
req.headers / req.get()	Request headers
req.path	Path only
req.url	Path + query string
req.originalUrl	Untouched original URL (even after router rewrites)

- **res methods:** `send()`, `json()`, `status()`, `sendStatus()`, `redirect()`, `set()`, `cookie()`, `render()`, `download()`, `sendFile()`. Most are chainable (return `res`).
- **Common gotcha:** calling `res.send()/json()` twice for one request throws *"Cannot set headers after they are sent"*. Always `return` after sending inside conditionals.
- `multipart/form-data` (file uploads) cannot be parsed by `express.json()/urlencoded()` – needs `multer`.

Code	Meaning
200 / 201 / 204	OK / Created / No Content
400 / 401 / 403	Bad Request / Unauthorized / Forbidden
404 / 409 / 422	Not Found / Conflict / Unprocessable Entity
500	Internal Server Error

5 Error Handling in Express

- Error-handling middleware needs **exactly 4 parameters**: (`err`, `req`, `res`, `next`), defined **last**, after all routes.
- **Sync errors** thrown inside a route handler are **caught automatically** by Express.
- **Async errors** (inside `async/Promises`) are **NOT** auto-caught in Express 4 – must use `try/catch` + `next(error)`, or a wrapper. Express 5 fixes this natively.

```
const catchAsync = (fn) => (req, res, next) => fn(req, res, next).catch(next);

app.get('/user/:id', catchAsync(async (req, res, next) => {
  const user = await User.findById(req.params.id);
  if (!user) return next(new AppError('User not found', 404));
  res.json(user);
}));

app.use((err, req, res, next) => { // catch-all 404 (place before error handler)
  next(new AppError('Cannot find ${req.originalUrl}', 404));
});

app.use((err, req, res, next) => { // centralized error handler
  res.status(err.statusCode || 500).json({ message: err.message });
});
```

- **Custom error class** (`AppError` extends `Error`) adds `statusCode` and `isOperational` to distinguish expected errors from bugs; don't leak raw messages for non-operational errors.
- Multiple error handlers possible; call `next(err)` to pass along (e.g., log then respond).
- `express-async-errors` package patches Express to auto-forward async errors.

6 REST API Design

- **REST** = architectural style: resource-based URLs (nouns, plural), correct HTTP methods, **statelessness**, client-server separation, uniform interface.
- URLs: `/users` not `/getUsers`; nesting for relationships (`/users/:id/orders`); filters via **query params**, not path.

Method	Idempotent?	Notes
GET	Yes	Read only
PUT	Yes	Replace entire resource (send all fields)
PATCH	Not guaranteed	Partial update (send only changed fields)
DELETE	Yes	Repeated delete = same end state
POST	No	Creates a new resource each call

- **Idempotent** = calling multiple times has the same effect as calling once.
- Version APIs: `/api/v1/...` to avoid breaking existing clients.
- Keep response shape consistent: `{ success, data }` / `{ success, message }`.
- REST (multiple endpoints, fixed shape) vs GraphQL (single endpoint, client picks fields) vs SOAP (XML, rigid, legacy enterprise).

7 Authentication & Authorization

- **Authentication** = who you are (login/identity). **Authorization** = what you're allowed to do (permissions). Authn always runs before authz.
- **Session-based**: server stores session (memory/Redis/DB), client holds a session-ID cookie. Stateful; needs shared store to scale across servers.
- **JWT (JSON Web Token)**: `header.payload.signature`. Self-contained, **stateless**, scales well. Harder to revoke early (use blacklist or short expiry + refresh tokens).
- Payload is only **base64-encoded, not encrypted** – never store secrets in it. Signature only proves it wasn't tampered with.

```
const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });

function verifyToken(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
    if (err) return res.status(403).json({ message: 'Invalid/expired token' });
    req.user = decoded;
    next();
  });
}
```

- **bcrypt**: `hash()` passwords (one-way, salted) before storing; `compare()` to verify at login. Never store plain text. Hashing $\neq$ encryption (encryption is reversible).
- Store JWT in **httpOnly cookie** (safe from XSS) rather than `localStorage` (readable by JS, vulnerable to XSS).
- **Access token** (short-lived) + **Refresh token** (long-lived, used to get new access tokens) pattern avoids frequent re-login.
- Authorization example: role-check middleware runs *after* `verifyToken`.

8 Database Integration (MongoDB + Mongoose)

- **SQL**: tables/rows, fixed schema, joins. **NoSQL (MongoDB)**: collections/documents (JSON-like), flexible schema.
- **Mongoose** = ODM: defines schema/validation on top of schema-less MongoDB, gives convenient query methods.
- **Schema** = structure/shape; **Model** (`mongoose.model()`) = used to actually query the DB.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true },
  role: { type: String, enum: ['user', 'admin'], default: 'user' }
}, { timestamps: true });
const User = mongoose.model('User', userSchema);
```

- CRUD: `create()/save()`, `find()/findById()/findOne()`, `findByIdAndUpdate(id, data, {new:true, runValidators:true})`, `findByIdAndDelete()`.
- `findByIdAndUpdate` returns the **old** doc by default – pass `{new:true}` for the updated one. Validators don't run on update by default – pass `runValidators:true`.
- **Embedding** (nested data, fast reads, duplication risk) vs **Referencing** + `.populate()` (`ObjectId` + `ref`, avoids duplication, extra query).

- **ValidationError** on bad data; **error code 11000** on duplicate key (**unique** violation).
- **Mongoose middleware/hooks**: `pre('save', ...)` / `post()` – e.g., hash password before saving.
- Filtering/sorting/pagination: `find(filter).sort().skip().limit()`. Operators: `$gte`, `$lte`, `$in`, `$or`, `$ne`.
- SQL equivalent tools: **Sequelize** / **Prisma** (ORMs).

9 CORS & Security

- **CORS** (Cross-Origin Resource Sharing): **browser** security feature controlling whether a page from one **origin** (protocol+domain+port) can access a server on another origin.
- Default browser rule: **Same-Origin Policy**. CORS is the controlled exception, set via server response headers.
- **CORS is enforced by the browser, not the server** – the server always processes the request; the browser hides the response from JS if not allowed. That's why Postman/curl never show CORS errors.
- **Preflight request**: browser auto-sends **OPTIONS** before "non-simple" requests (**PUT/DELETE**/custom headers) to check permission first.
- Key headers: **Access-Control-Allow-Origin**, **-Methods**, **-Headers**, **-Credentials**.

```
app.use(cors({
  origin: 'https://myfrontend.com',
  credentials: true // cannot combine credentials:true with origin:'*'
}));
```

- **Security stack**: `helmet()` (secure headers), `cors()` (restricted origin), `express-rate-limit` (brute-force/DoS protection, stricter on `/login`), `express-validator/Joi` (input validation), `express-mongo-sanitize` (NoSQL injection), body size limit (`express.json({limit: '10kb'})`), **HTTPS** in production.

10 File Uploads & Validation

- File uploads use **Content-Type**: `multipart/form-data`, which `express.json()/urlencoded()` cannot parse – use **Multer**.

```
const upload = multer({
  storage: multer.diskStorage({
    destination: (req, file, cb) => cb(null, 'uploads/'),
    filename: (req, file, cb) => cb(null, Date.now() + '-' + file.originalname)
  }),
  fileFilter: (req, file, cb) => cb(null, ['image/jpeg', 'image/png'].includes(file.mimetype)),
  limits: { fileSize: 5*1024*1024 }
});
app.post('/profile', upload.single('avatar'), (req, res) => { ... }); // req.file
```

- `upload.single(field)` → `req.file`; `upload.array(field, max)` → `req.files` (same field); `upload.fields([...])` → `req.files` (different fields).
- `diskStorage` (save to disk, custom filenames to avoid overwrite) vs `memoryStorage` (buffer, good for cloud upload e.g. S3/Cloudinary).
- `fileFilter` + `limits.fileSize` restrict type/size. `mimetype` can be spoofed by the client – not fully secure alone.
- **Validation libraries**: `express-validator` (`body()`, `param()`, `query()`, `validationResult()`) or `Joi` (schema-based, like `Mongoose` style).
- Validation = checks rules; Sanitization = cleans/modifies input (trim, escape).
- Multer errors (`MulterError`) caught via wrapped middleware or the centralized error handler.
- Production: upload to **cloud storage** (S3, Cloudinary) instead of local disk (servers restart/scale).

11 Advanced / Good-to-Know

MVC

- **Model** (schema/DB logic) – **Controller** (business logic) – **Routes** (URL mapping). APIs skip a traditional "View," return JSON instead.
- Benefits: testable, maintainable, team-friendly.

REST Best Practices (Checklist)

- Nouns in URLs, correct methods/status codes, statelessness, versioning (`/api/v1`), plural names, consistent response shape, filtering/sorting/pagination via query params, centralized error handling, documented (Swagger/OpenAPI), secured (auth, rate limit, validation, CORS).

Pagination, Filtering, Sorting

```
const { category, sort, page=1, limit=10 } = req.query;
const filter = category ? { category } : {};
const products = await Product.find(filter)
  .sort(sort?.split(',').join(' ') || '-createdAt')
  .skip((page-1)*limit).limit(Number(limit));
```

- **Offset-based** (page/limit, simple, can shift if data changes) vs **Cursor-based** (reference point like last ID, stable for large/changing datasets).

Rate Limiting & Caching

- **Rate limiting** protects against abuse/brute-force/DoS (**express-rate-limit**).
- **In-memory cache** (simple, single-server, e.g. **Map**) vs **Redis** (shared across instances/cluster workers – production-grade).
- **HTTP caching**: **Cache-Control**, **ETag** headers let browsers/CDNs cache without hitting the server.

WebSockets (Real-Time)

- **REST** = request-response only; doesn't fit real-time push (chat, live notifications). **WebSockets** give a persistent two-way connection.
- **Socket.io**: **io.on('connection', socket=>{...})**; **socket.emit()** (to one client) vs **io.emit()** (broadcast to all); **socket.on(event, cb)** to listen.